Unmanned driving integrated AI large model

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

This tutorial uses the factory-installed road sign and lane recognition models, primarily implementing autonomous driving road sign detection + map navigation + multimodal large model for intelligent control.

2. Course Prerequisites

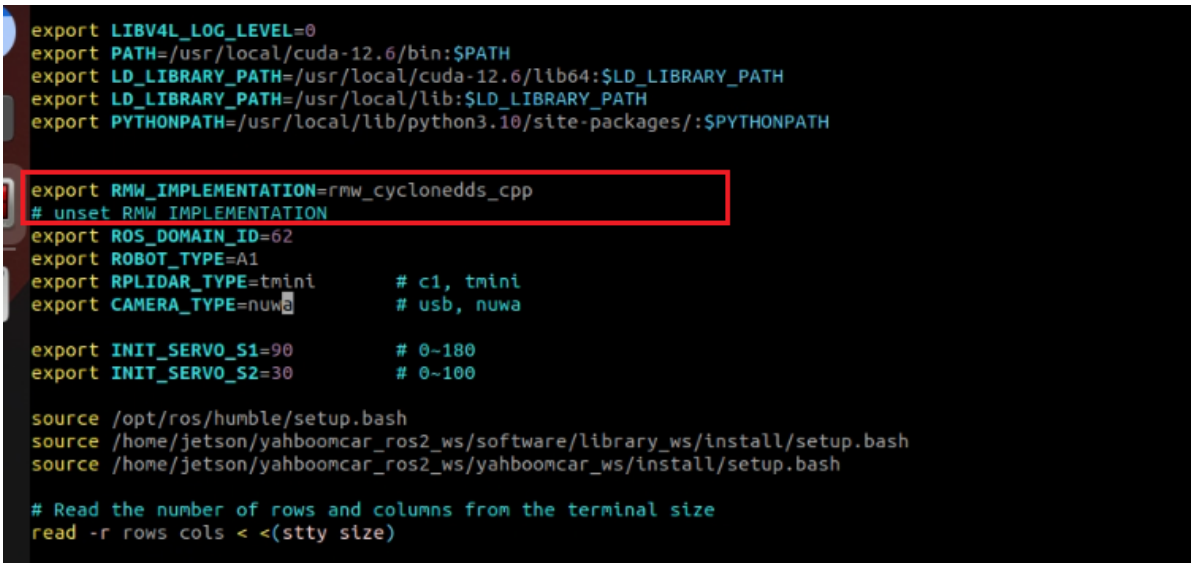
- After completing the "Road Network Planning" chapter, save the road network navigation map and the road network map showing the car's trajectory.
- After completing the "AI Large Model Fundamentals" chapter, configure the large model's api_key and app_id.
- Obtain the pose data of the points the car needs to reach and store it in the large model's map_mapping.yaml file.
- Modify to road network navigation communication mode.

3. Configuration File Path

- Road Network Navigation Communication Mode

Open ~/.bashrc, uncomment this line, and change it as shown in the image. (After learning about the road network planning function, please restore this comment to avoid affecting the previous examples.)

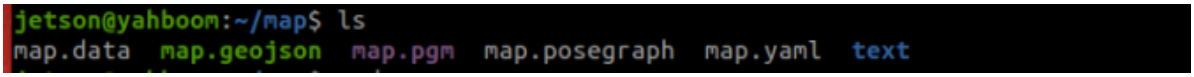
```
vim ~/.bashrc
:wq #Save and exit
source ~/.bashrc #Update environment
```



- Road network navigation map and road network map

```
~/map
```

The image shows the saved map and road network diagram. The map.geojson file contains the road network diagram of the car's trajectory.

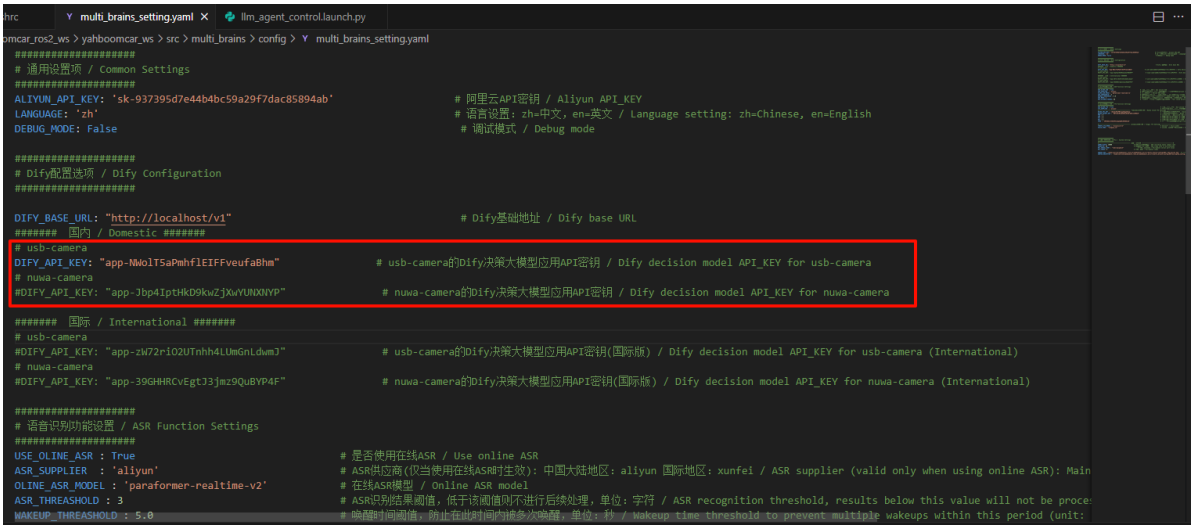


- Configure AI large model parameters

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/multi_brains_setting.yaml
```

multi_brains_setting.yaml:

The main configuration for this yaml file is the application body ID. Based on the camera model, select the corresponding application body key and disable the other one. After configuration, you need to recompile and run it. For the large AI model, you need to open an account and bind a key on the API platform beforehand. For specific configuration details, please refer to section 5, "Configuring the Large AI Model," in the Basic Knowledge chapter.



- Configure the pose data of the target points

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/map_mapping.yaml
```

Add new target points under the route navigation section, as this course uses route navigation.

```yaml
#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致
#Here, you can add the raster map coordinate points corresponding to the added area. Please note that the format should be consistent with the above
road_net_map_areas: #路网导航 road network navigation
  A:
    name: '五金店'
    position:
      x: 1.1409525871276855
      y: -0.6988544464111328
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.7420813339272786
      w: 0.6703098491270368

  B:
    name: '教学楼'
    position:
      x: 2.229278564453125
      y: -0.5782679915428162
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.999210671704489
      w: 0.03972447044157906

  C:
    name: '学校'
    position:
      x: 1.520
      y: 0.124
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.093
      w: 0.996

  D:
    name: '工地'
    position:
      x: 0.006
      y: -1.546
      z: 0.0
    orientation:
      x: 0.0
```

If you don't know how to obtain the pose data of a point, you can first refer to the "04-Route Planning Combined with Sandbox Map Case" in the route planning navigation section.

After modifying the yaml files, you need to recompile and then run the program.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
colcon build --packages-select multi_brains
```

4. Program Startup

4.1 Startup Command

- Open the host terminal and enable the Dify environment.

```
sh ~/bringup_dify.sh
```

```
                                    [System Information]
IP_Address_1: 192.168.11.108
IP_Address_2: 172.18.0.1
------------------------------------------------------
ROS_DOMAIN_ID: 63 | ROS: humble
my_robot_type: M1 | my_lidar: c1 | my_camera: usb
------------------------------------------------------
sunrise@ubuntu:~$ sh ~/bringup_dify.sh
[+] Running 11/11
 ✔ Container docker-ssrf_proxy-1     Running                                    0.0s
 ✔ Container docker-web-1            Running                                    0.0s
 ✔ Container docker-redis-1          Running                                    0.0s
 ✔ Container docker-weaviate-1       Running                                    0.0s
 ✔ Container docker-db-1             Healthy                                    0.5s
 ✔ Container docker-sandbox-1        Running                                    0.0s
 ✔ Container docker-api-1            Running                                    0.0s
 ✔ Container docker-worker-1         Running                                    0.0s
 ✔ Container docker-worker_beat-1    Running                                    0.0s
 ✔ Container docker-plugin_daemon-1  Running                                    0.0s
 ✔ Container docker-nginx-1          Running                                    0.0s
```

- Launch large model + chassis + camera

```
ros2 launch multi_brains llm_agent_control.launch.py enable_route_nav:=True
```

The following content will be displayed after initialization is complete

```
[action_service_nuwa-15]    from pkg_resources import resource_stream, resource_exists
[action_service_nuwa-15] [INFO] [1766129486.356630338] [action_service_nuwa]: enable_route_nav: False
[action_service_nuwa-15] [INFO] [1766129486.726464447] [action_service_nuwa]: ROS_Action_Service Initialization completed
[model_service-14] [INFO] [1766129518.715920692] [model_service]: Dify LLM connection successful
[model_service-14] [WARN] [1766129518.719145287] [rcl.logging_rosout]: Publisher already registered for provided node name. I
f this is due to multiple nodes with the same name then all logs for that logger name will go out over the existing publisher
. As soon as any node with that name is destructed it will unregister the publisher, preventing any further logs for that nam
e from being published on the rosout topic.
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.rear
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.center_lfe
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.side
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.hdmi
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.hdmi
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.modem
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.modem
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.phoneline
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.phoneline
[model_service-14] ALSA lib pcm_oss.c:397:(_snd_pcm_oss_open) Cannot open device /dev/dsp
[model_service-14] ALSA lib pcm_oss.c:397:(_snd_pcm_oss_open) Cannot open device /dev/dsp
[model_service-14] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model_service-14] ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
[model_service-14] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model_service-14] ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
[model_service-14] [INFO] [1766129519.225800065] [ASR_Detect]: The online ASR_Engine:paraformer-realtime-v2 initialization ha
s been completed
[model_service-14] [INFO] [1766129519.274575027] [model_service]: Tongyi TTS_Engine initialized successfully
[model_service-14] [INFO] [1766129519.277227745] [model_service]: ROS_LLM_Service Initialization completed
```

- Enter the road network navigation docker

```
bringup_ros2kilted
```

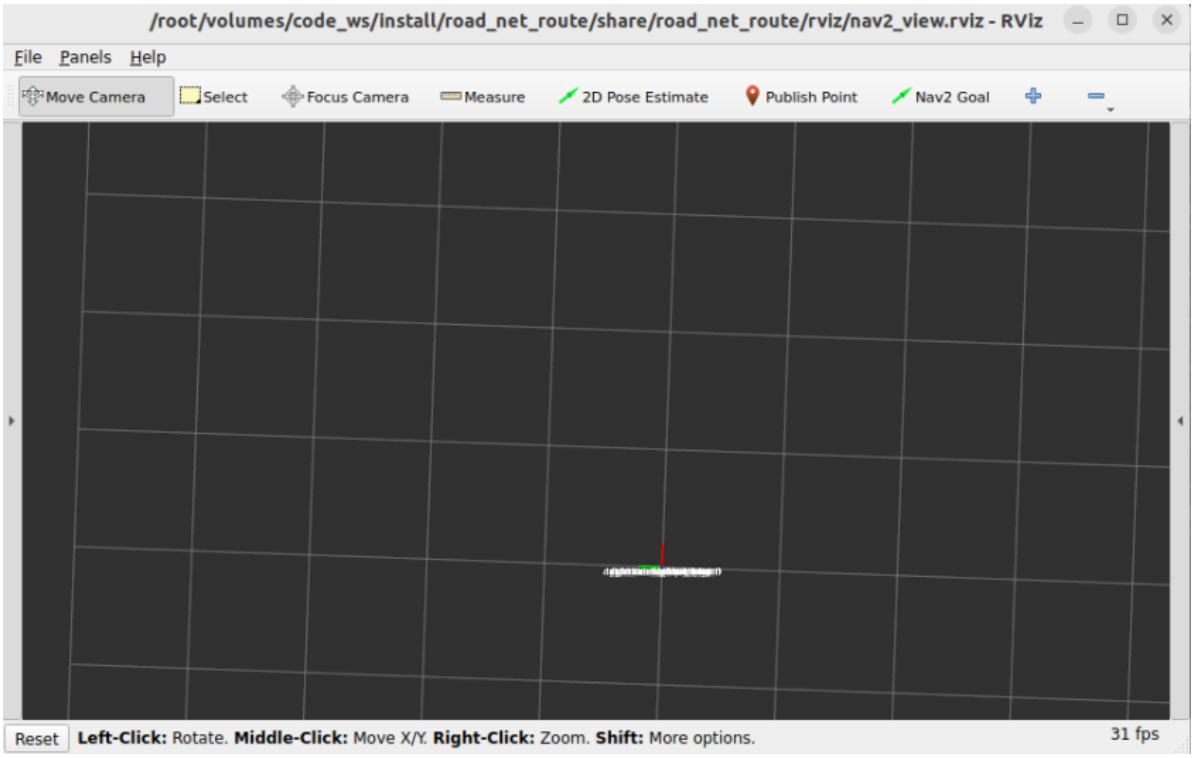
```
etson@yahboom:~$ bringup_ros2kilted
ccess control disabled, clients can connect from any host
+] Running 1/1
✔ Container ros2_kilted-ros2_kilted-1  Started                    1.8s
etson@yahboom:~$
```

```
exec_ros2kilted
```

```
jetson@yahboom:~$ exec_ros2kilted
root@yahboom:/#
```

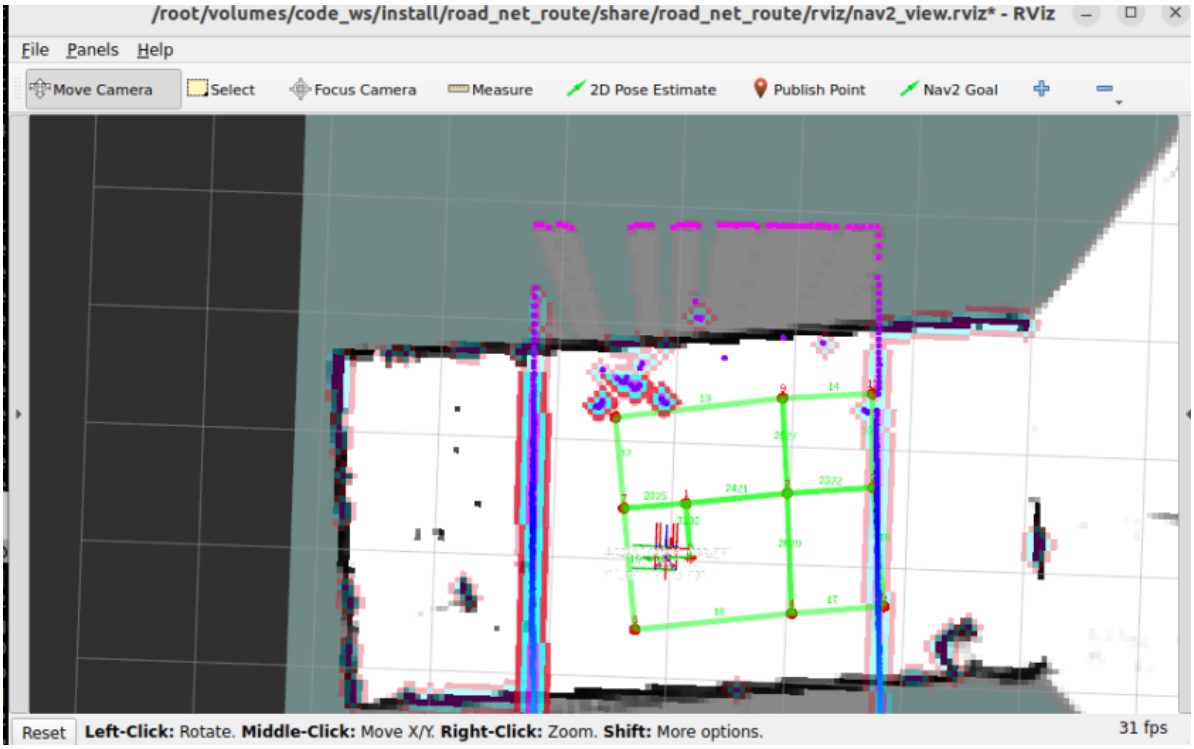
Enter this command in the terminal.

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

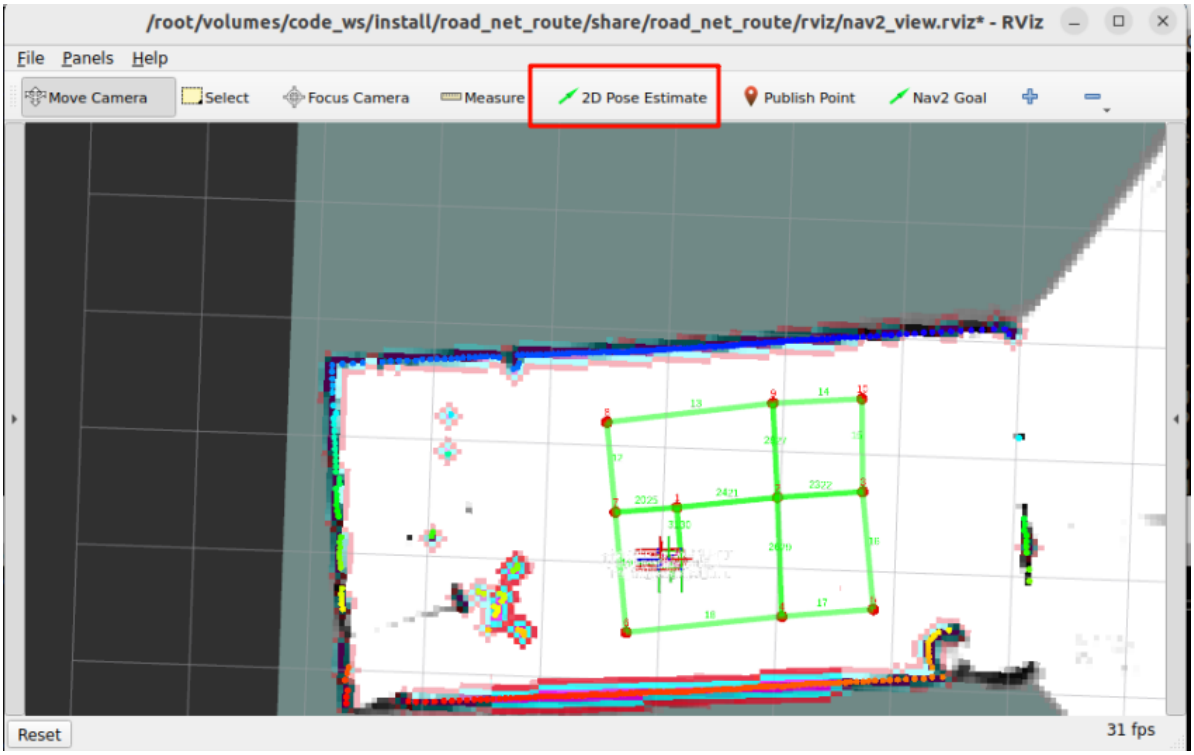


- (On-board) Launch **cartograph** for quick location.

```
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/yahboomcar.pbstream
```



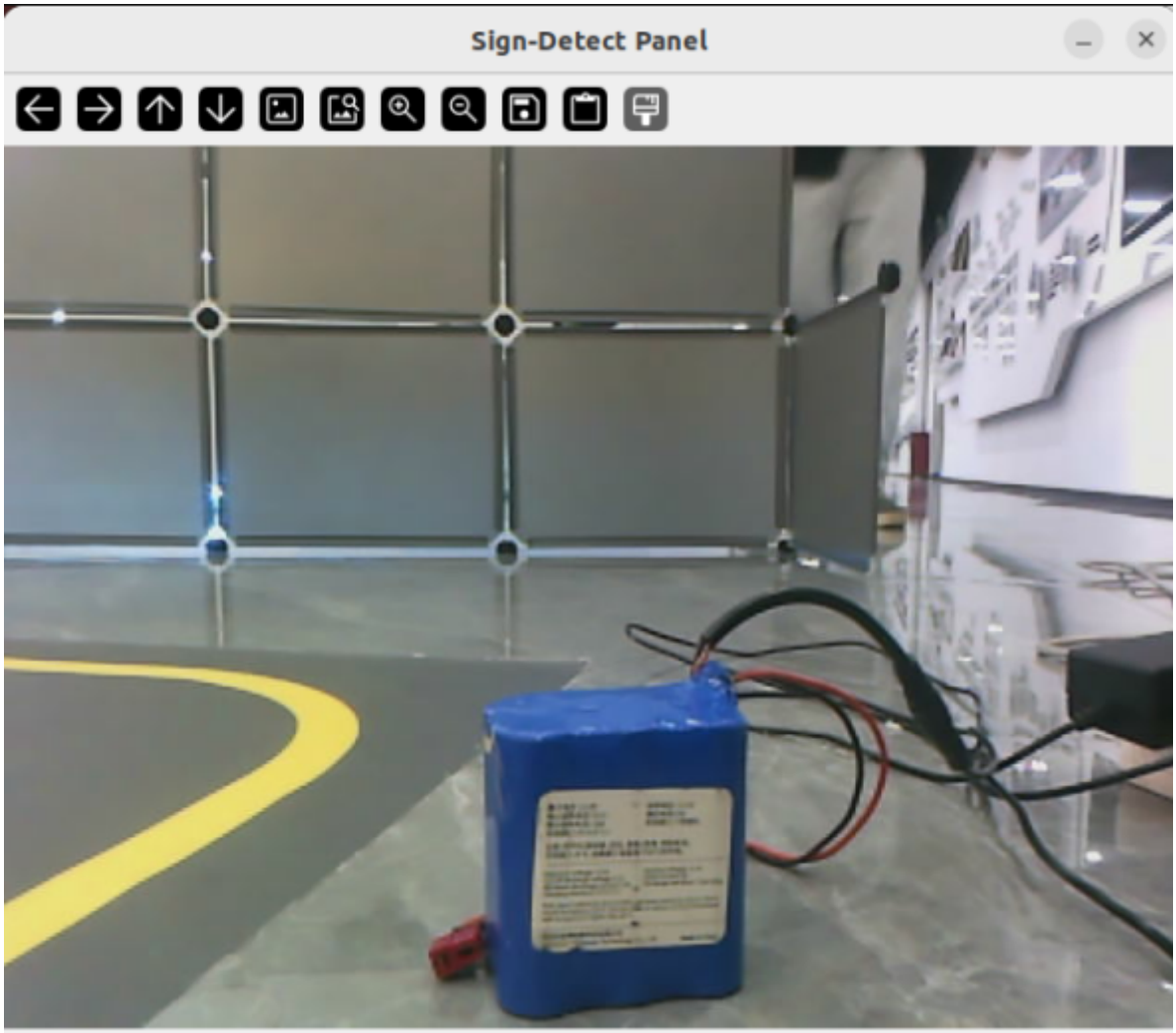
Upon entering, you'll notice that the point cloud and the map don't overlap; you'll need to use 2D Pose Estimate to reposition the car.



- Start YOLO Road Sign Recognition

```
ros2 launch auto_drive auto_drive.launch.py enable_route_nav:=True
```

After successful launch, a visual interface will appear.



4.2 Test Cases

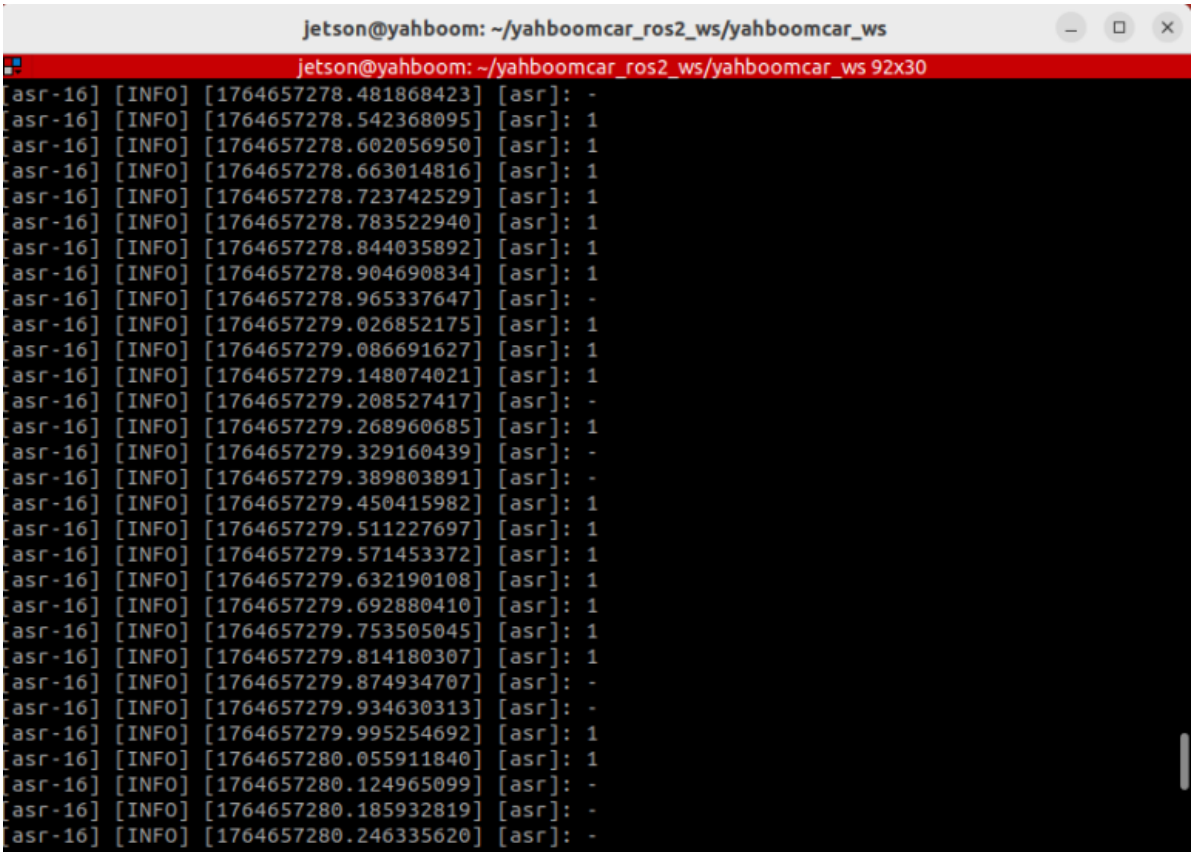
Here are some reference test cases. Users can create their own dialogue commands.

- Navigate to the construction site while following traffic rules

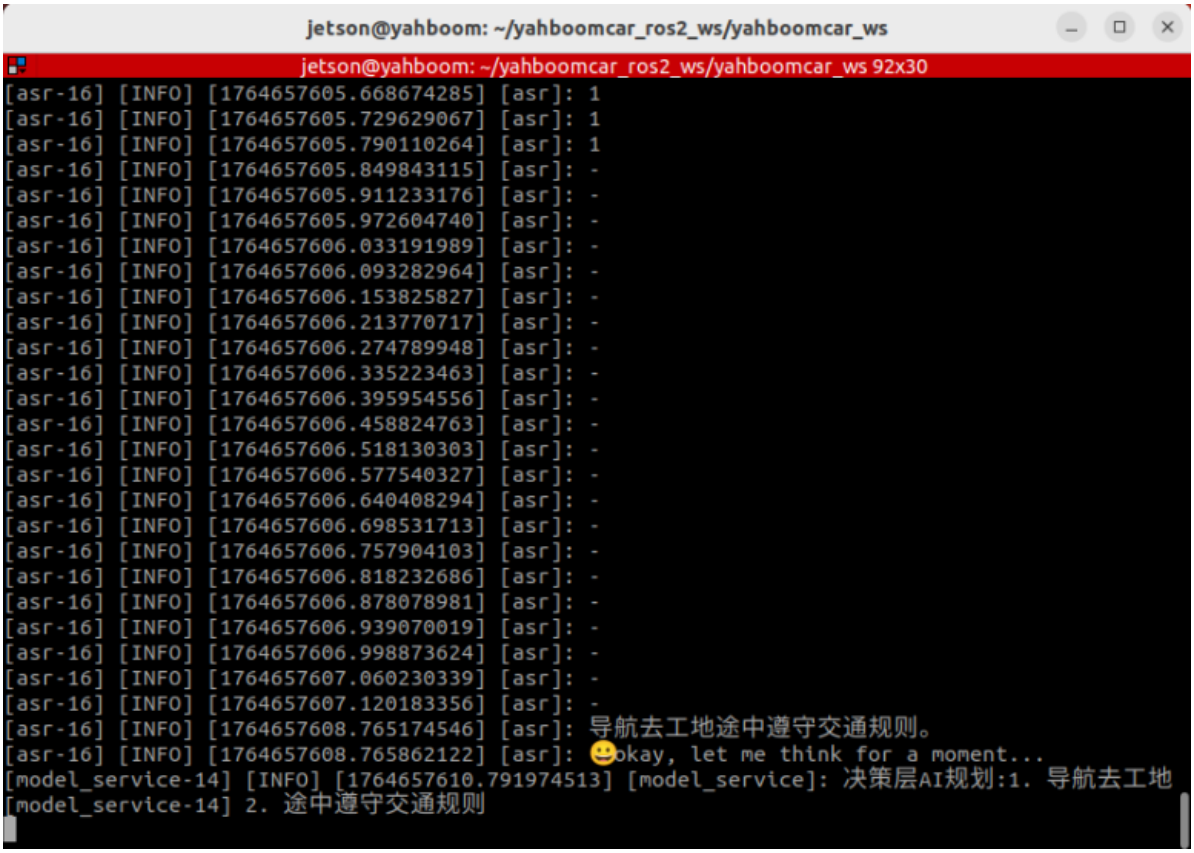
4.2.1 Case 1

First, wake up the robot with "Hello yahboom". The robot will respond with "I'm here, please give instructions". After the robot responds, the buzzer will sound briefly (beep—), then you can speak. The robot will perform voice activity detection (VAD); if there is voice activity, it will print 1, otherwise it will print -. When speaking ends, it will perform tail silence detection; if silent for more than 450ms, it will stop recording.

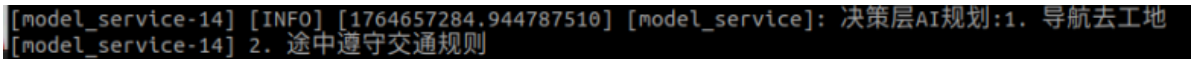
Dynamic voice activity detection (VAD) is shown in the figure below:



The robot first recognizes the user's speech, then makes decisions and takes actions according to the instructions. Simultaneously, the terminal prints the following information:



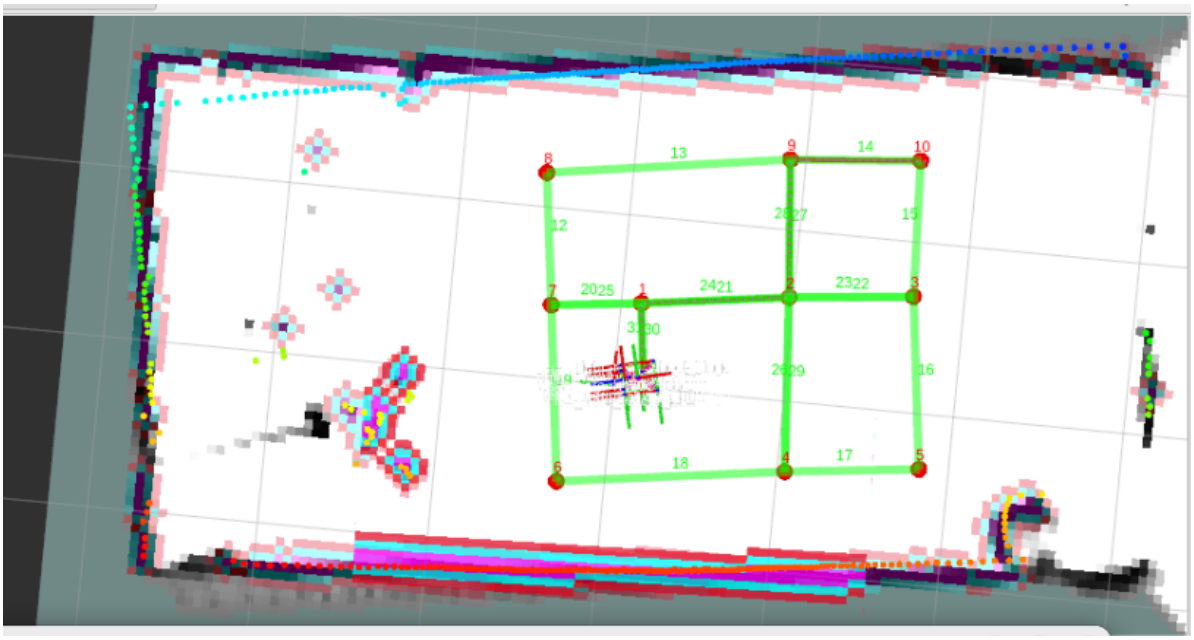
The decision-making layer's large model outputs the planned task steps:



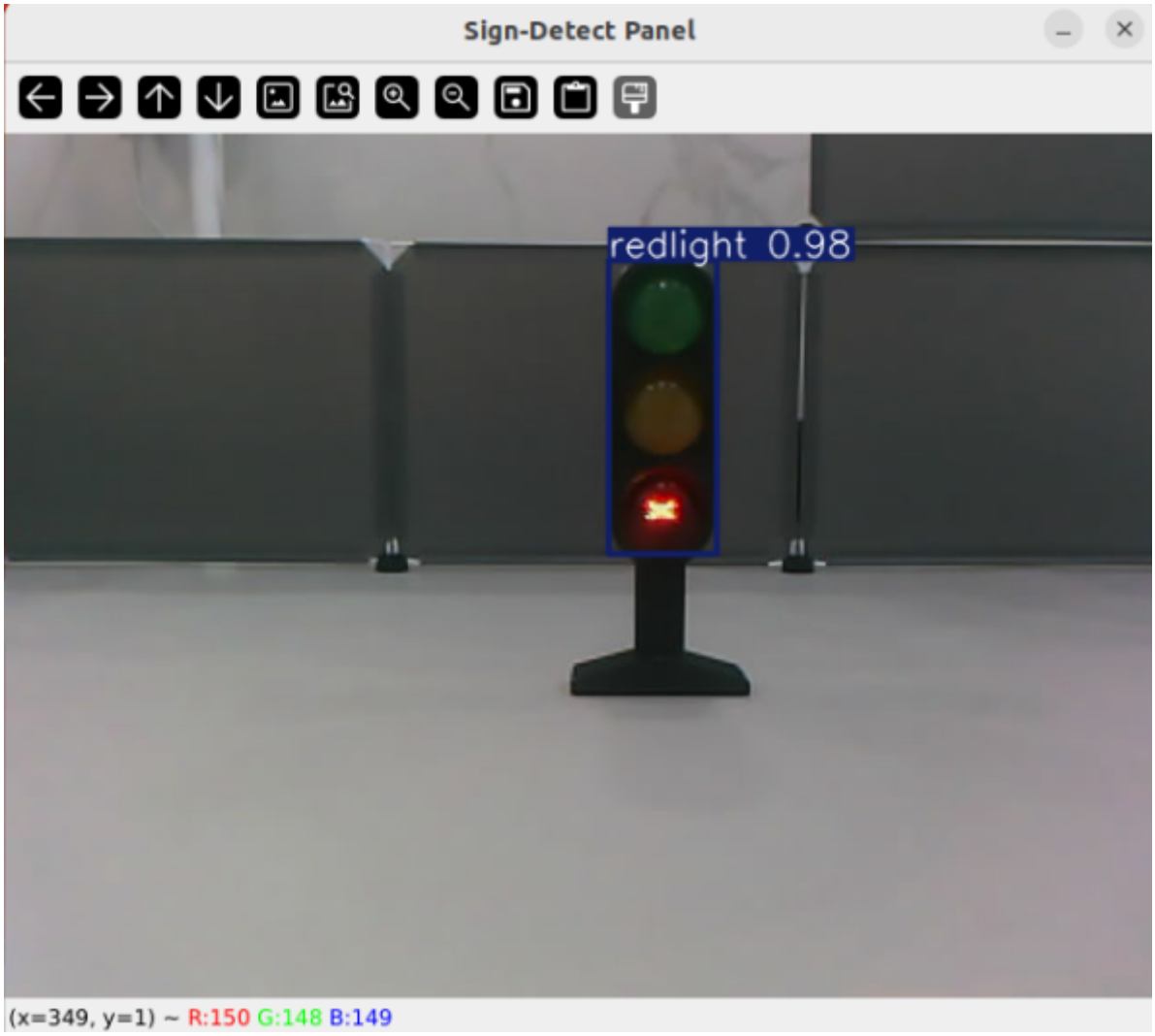
Then, the execution layer's large model executes according to these task steps:

```
[action_service_nuwa-15] [INFO] [1764658284.847215058] [action_service]: Published message:
机器人反馈:执行control_sign(enable)完成
[action_service_nuwa-15] [INFO] [1764658284.858613098] [action_service]: Published message:
机器人反馈：执行['navigation(D)', 'control_sign(enable)']完成
[model_service-14] [INFO] [1764658286.053808441] [model_service]: "action": [], "response":
交通规则已开启，现在安全出发去工地
[model_service-14] [INFO] [1764658287.629069342] [model_service]: "action": ['finishtask()']
, "response": 已到达工地，任务完成!
[action_service_nuwa-15] [INFO] [1764658291.935198082] [action_service]: Published message:
机器人反馈：回复用户完成
[model_service-14] [INFO] [1764658293.260168282] [model_service]: "action": ['finishtask()']
, "response": 已到达工地，任务完成!
```

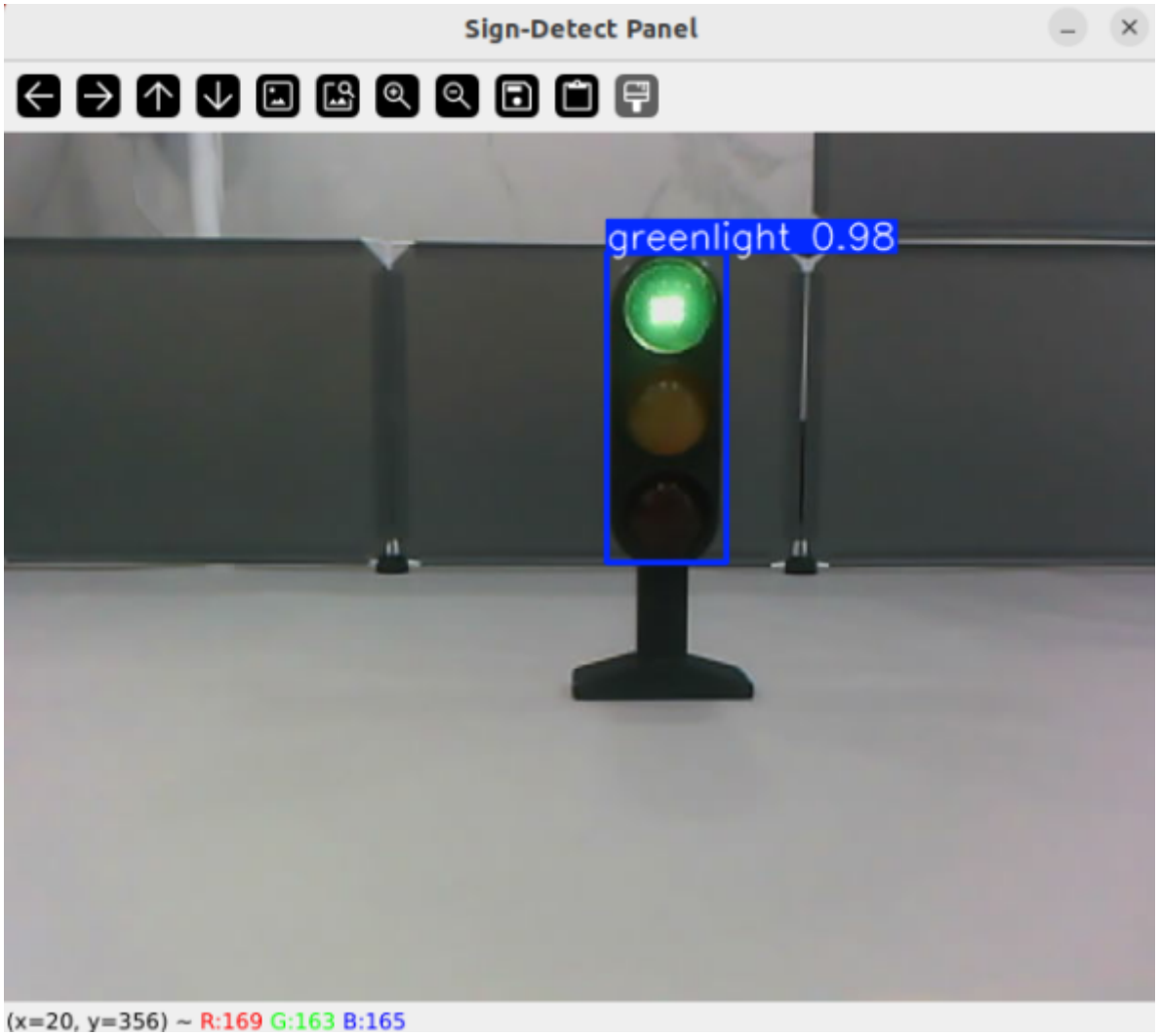
Simultaneously, as you can see, the road network navigation interface displays a grid route for the car to navigate to the construction site.



If we turn on a red light midway through the navigation, the car will stop and wait for the green light before resuming navigation.



When the green light comes on, the car will continue navigating. **Note that the traffic lights should not be placed too close to the car's route; if they are too close, they may be perceived as obstacles and affect navigation.**



5. Source Code Analysis

5.1 Path to the Large Model Source Code

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
```

action_service_nuwa.py: This file contains the main logic for the large model integration.

Key functionalities include:

- Providing action servers to handle navigation and robot control commands
- Integrating speech recognition and speech synthesis
- Enables switching between road network navigation and regular navigation
- Handles large language model interactions

It supports two navigation modes: **Regular Navigation**: a standard navigation mode based on a single point target; and **Road Network Navigation**: a navigation mode based on a predefined road network path. To integrate with autonomous driving, `enable_route_nav: True` needs to be enabled for road network navigation.

Navigation Process:

- Receive navigation request
- Select navigation strategy based on current mode (Road Network/Regular)
- Execute navigation actions
- Provide feedback and results

```
# In the init_param_config method
self.declare_parameter("enable_route_nav", False)
self.enable_route_nav =
self.get_parameter("enable_route_nav").get_parameter_value().bool_value
```

Navigation mode switching:

```
def handle_navigation_request(self, destination):
    """Handling navigation requests"""
    if self.enable_route_nav:
        # Road network navigation mode
        if destination in self.route_network:
            self.follow_route(self.route_network[destination])
        else:
            self.get_logger().warn(f"No route found for destination:
{destination}")
    else:
        # Normal navigation mode
        self.navigate_to_waypoint(destination)
```

5.2 Road Network Navigation Source Code Path

```
#jetson orin nano
~/ros2_kilted/volumes/code_ws/src/road_net_route/launch/roadnet_nav2.launch.py
#RDK X5
/home/sunrise/ros2_kilted/volumes/code_ws/src/road_net_route/launch/roadnet_nav2
.launch.py
```

roadnet_nav2.launch.py:

The launch file will directly call the file saved in the /root/map path. If it has a different name, you can change the name here and then recompile and run it.

```
declare_map_yaml_cmd = DeclareLaunchArgument(
        'map',
        default_value='/root/map/map.yaml',
    )
    declare_pose_map_cmd = DeclareLaunchArgument(
        'pose_map',
        default_value='/root/map/map',
    )
    declare_roadnet_file_cmd = DeclareLaunchArgument(
        'roadnet_file',
        default_value='/root/map/map.geojson',
            description=".geojson格式路网文件路径",
    )
```

The parameter file used for navigation.

```
declare_params_file_cmd = DeclareLaunchArgument(
        'params_file',

 default_value=os.path.join(get_package_share_directory('road_net_route'),
'config', 'nav2_kilted.yaml'),
        description='Full path to the ROS2 parameters file to use for all
launched nodes',
    )
```

You can open the nav2_kilted.yaml file in this path and modify some navigation parameters. Normally, you don't need to modify the parameters; most of them are pre-tuned and don't require modification.

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted.yaml
```

5.3 YOLO recognition source code

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/launch/auto_drive.launch.py
```

auto_drive.launch.py:

The YOLO system is used to recognize road signs and instruct the car to perform corresponding actions. However, because this is based on a road network navigation system, the right turn and stop road signs will be disabled, as these two signs will interfere with the road network navigation.